

A [queue](#) is an abstract data type that maintains the order in which elements were added to it, allowing the oldest elements to be removed from the front and new elements to be added to the rear. This is called a *First-In-First-Out* (FIFO) data structure because the first element added to the queue (i.e., the one that has been waiting the longest) is always the first one to be removed.

A basic queue has the following operations:

- *Enqueue*: add a new element to the end of the queue.
- *Dequeue*: remove the element from the front of the queue and return it.

In this challenge, you must first implement a queue using *two stacks*. Then process q queries, where each query is one of the following **3** types:

1. x : Enqueue element x into the end of the queue.
2. Dequeue the element at the front of the queue.
3. Print the element at the front of the queue.

Input Format

The first line contains a single integer, q , denoting the number of queries.

Each line i of the q subsequent lines contains a single query in the form described in the problem statement above. All three queries start with an integer denoting the query *type*, but only query **1** is followed by an additional space-separated value, x , denoting the value to be enqueued.

Constraints

- $1 \leq q \leq 10^5$
- $1 \leq \text{type} \leq 3$
- $1 \leq |x| \leq 10^9$
- It is guaranteed that a valid answer always exists for each query of type **3**.

Output Format

For each query of type **3**, print the value of the element at the front of the queue on a new line.

Sample Input

STDIN	Function
10	$q = 10$ (number of queries)
1 42	1st query, enqueue 42
2	dequeue front element
1 14	enqueue 42
3	print the front element
1 28	enqueue 28
3	print the front element
1 60	enqueue 60
1 78	enqueue 78
2	dequeue front element
2	dequeue front element

Sample Output

```
14
14
```

Explanation

Perform the following sequence of actions:

1. Enqueue **42**; *queue* = {**42**}.
2. Dequeue the value at the head of the queue, **42**; *queue* = {}.
3. Enqueue **14**; *queue* = {**14**}.
4. Print the value at the head of the queue, **14**; *queue* = {**14**}.
5. Enqueue **28**; *queue* = {**14**, **28**}.
6. Print the value at the head of the queue, **14**; *queue* = {**14**, **28**}.
7. Enqueue **60**; *queue* = {**14**, **28**, **60**}.
8. Enqueue **78**; *queue* = {**14**, **28**, **60**, **78**}.
9. Dequeue the value at the head of the queue, **14**; *queue* = {**28**, **60**, **78**}.
10. Dequeue the value at the head of the queue, **28**; *queue* = {**60**, **78**}.